

Software Project Planning

Software Project Planning is the process of defining the scope, objectives, tasks, resources, schedule, and risks of a software development project in advance, to ensure that it can be completed within time, budget, and quality constraints.

Objectives of Project Planning

- Scope definition
- Cost, effort, time estimation
- Resource allocation
- Scheduling & milestones
- Risk identification & control
- Quality assurance

Steps in Project Planning

1. Scope Definition

- Boundaries, goals, exclusions
- Example: Online shopping system

2. Effort, Cost, and Time Estimation

- Techniques: LOC, Function Point Analysis, COCOMO
- Comparison of estimation models

3. Resource Planning

- Human resources, hardware, software
- Resource Allocation Matrix

4. Project Scheduling

- Work Breakdown Structure (WBS)
- Gantt charts
- PERT/CPM (critical path analysis)

5. Risk Management

- Types of risks (technical, schedule, cost, resource)
- Risk identification, analysis, mitigation, monitoring

6. Quality Planning

- Standards (ISO, CMMI)
- Metrics: defect density, reliability, usability

7. Documentation

- Project Plan Document (PPD) structure

Estimation Models

- LOC-based estimation
- Function Point Analysis (FPA)
- COCOMO (Basic, Intermediate, Detailed)
- Example calculations with sample projects

Project Scheduling Tools

- Gantt chart with figure
- PERT chart with critical path example
- Comparison between Gantt vs PERT
- Case study: Online Library Management scheduling

Importance of Project Planning

- Preventing overruns
- Customer satisfaction
- Resource optimization
- Lessons from real-world failed projects

Software Architecture (SA):

Software Architecture is the high-level structure of a software system, which defines its components, the relationships between them, and the principles guiding its design and evolution.

It acts as a blueprint for both the system and the project team, ensuring that the software meets functional requirements (what the system should do) and non-functional requirements (performance, security, scalability, reliability, etc.).

Objectives of Software Architecture

- Ensure quality attributes
- Support scalability, maintainability
- Enable stakeholder communication
- Reusability

Components of Software Architecture

- Modules
- Components & connectors
- Interfaces
- Data flow

Architectural Styles

1. Data-centered (repository, blackboard)
2. Data-flow (pipe and filter, batch sequential)
3. Call-and-return (main-subroutine, layered)
4. Object-oriented
5. Event-driven (publish-subscribe)
6. Client-server
7. Service-Oriented Architecture (SOA)
8. Microservices (modern evolution of SOA)

Design Patterns

- Creational (Singleton, Factory)
- Structural (Adapter, Composite)
- Behavioral (Observer, Strategy)
- Examples with diagrams

Quality Attributes

- Performance
- Reliability
- Usability
- Maintainability
- Security
- Scalability
- Trade-offs between attributes

Architectural Evaluation

- SAAM (Software Architecture Analysis Method)
- ATAM (Architecture Trade-off Analysis Method)
- Cost-benefit trade-offs

Documentation

- IEEE 1471 standard
- UML diagrams (component, deployment, sequence, package diagrams)
- Architecture Decision Records (ADR)

Importance of Software Architecture

- Managing complexity
- Supporting system evolution
- Enabling quality attributes
- Industry examples (Google, Amazon, Netflix)

Software Design and Implementation:

After requirements analysis and specification, the next step in the Software Development Life Cycle (SDLC) is Software Design, followed by Implementation (Coding).

- Software Design → The process of transforming requirements into a blueprint for building the system.
- Implementation → The process of writing code and realizing the design into a working software product.

Software Design:

Software Design is the process of defining the architecture, components, interfaces, and data for a system to satisfy specified requirements.

Objectives of Software Design

- To create a blueprint for implementation.
- To ensure system satisfies functional and non-functional requirements.
- To manage complexity through modularization.
- To enable reusability, maintainability, scalability, and reliability.

Levels of Design

1. High-Level Design (Architectural Design)

- Focuses on **system architecture**.
- Defines modules, components, subsystems, and their interactions.
- Example: Layered architecture, client-server, microservices.

2. Low-Level Design (Detailed Design)

- Focuses on **internal logic** of components.
- Describes algorithms, data structures, database schemas, and class diagrams.
- Example: Designing methods in a class, writing pseudo-code.

Design Principles

- **Modularity** → Divide system into independent modules.
- **Abstraction** → Focus on essential details, hide implementation.
- **Encapsulation** → Bind data and methods together.

- **Separation of Concerns** → Divide system based on responsibilities.
- **Coupling and Cohesion:**
 - Low coupling = less dependency between modules.
 - High cohesion = module focuses on a single responsibility.

Design Methods

- Structured Design (DFDs, ER diagrams, flowcharts).
- Object-Oriented Design (OOD) (UML diagrams, classes, objects, inheritance, polymorphism).
- Component-Based Design.

Software Design Tools

- UML Diagrams:
 - Class Diagram
 - Use Case Diagram
 - Sequence Diagram
 - Component Diagram
 - Deployment Diagram

Software Implementation:

Software Implementation (or Coding) is the stage where the design is translated into source code using a programming language.

Objectives of Implementation

- To convert design into working software.
- To follow **coding standards and guidelines**.
- To ensure correctness, efficiency, and maintainability.

- To support future **testing, debugging, and maintenance**.

Programming Paradigms

- Procedural Programming (C, Pascal).
- Object-Oriented Programming (Java, C++, Python).
- Functional Programming (Haskell, Scala, JavaScript functional parts).

Implementation Process

1. Select suitable programming language & environment.
2. Write code for modules.
3. Unit testing of individual modules.
4. Integrate modules together.
5. Perform system testing.

Tools for Implementation

- IDEs (Integrated Development Environments) → Eclipse, IntelliJ, PyCharm, Visual Studio.
- Version Control Systems → Git, GitHub, GitLab.
- Build Tools → Maven, Gradle, Make.
- Continuous Integration Tools → Jenkins, Travis CI.

Challenges in Implementation

- Requirement changes during coding.
- Bugs due to logic or integration issues.
- Performance bottlenecks.
- Managing large codebases.

Example

Case: Online Shopping System

- **Design Phase:**

- High-Level: 3-tier architecture (Presentation layer, Business logic, Database).
- Low-Level: Class design (User, Cart, Product, Order).

- **Implementation Phase:**

- Code classes in Java/Python.
- Connect to database (MySQL, MongoDB).
- Develop UI (React/Angular).